Implementation and Modeling of a Processor

1 Introduction

We begin with the instruction set architecture of MIPS32, a 32-bit RISC processor that operates on 32 bits of data at a time. The note introduces its instruction formats, instruction execution cycle, pipelined datapath, and a behavioral Verilog model with test programs.

1.1 MIPS32 overview

MIPS32 has 32 general-purpose 32-bit registers, R0 through R31. Register R0 always contains zero and cannot be written. The 32-bit program counter (PC) points to the next instruction to fetch. MIPS uses a small set of addressing modes, including register, immediate, and register-indexed addressing. Only load and store instructions access memory. The examples assume a 32-bit memory word. The subset used in these notes includes the following load/store and register-register operations:

Listing 1: Instruction subset: memory and register operations

```
LW   R2, 124(R8)       // R2 = Mem[R8 + 124]
SW   R5, -10(R25)      // Mem[R25 - 10] = R5

ADD  R1, R2, R3        // R1 = R2 + R3
ADD  R1, R2, R0        // R1 = R2 + 0
SUB  R12, R10, R8      // R12 = R10 - R8
AND  R20, R1, R5       // R20 = R1 & R5
OR   R11, R5, R6       // R11 = R5 | R6
MUL  R5, R6, R7        // R5 = R6 * R7
SLT  R5, R11, R12      // R5 = (R11 < R12) ? 1 : 0
```

2 Instruction formats

MIPS32 instructions use three fixed 32-bit formats: R-type (register), I-type (immediate), and J-type (jump). Their fields have fixed positions, making instruction decoding straightforward even though not every format uses every field.

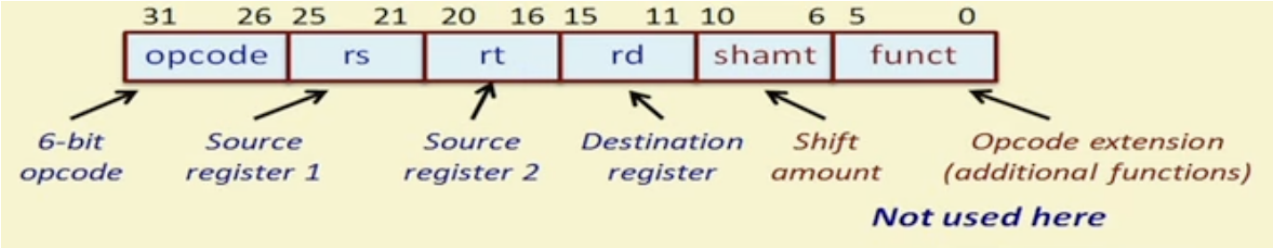
2.1 R-type instructions

An R-type instruction can name two source registers and one destination register. Shift instructions can also specify a shift amount. The format diagram is retained below. The instruction subset also includes immediate arithmetic, conditional branches, an unconditional jump, and halt:

Listing 2: Immediate, branch, jump, and halt instructions

```
ADDI  R1, R2, 25       // R1 = R2 + 25
SUBI  R5, R1, 150      // R5 = R1 - 150
SLTI  R2, R10, 10      // R2 = (R10 < 10) ? 1 : 0

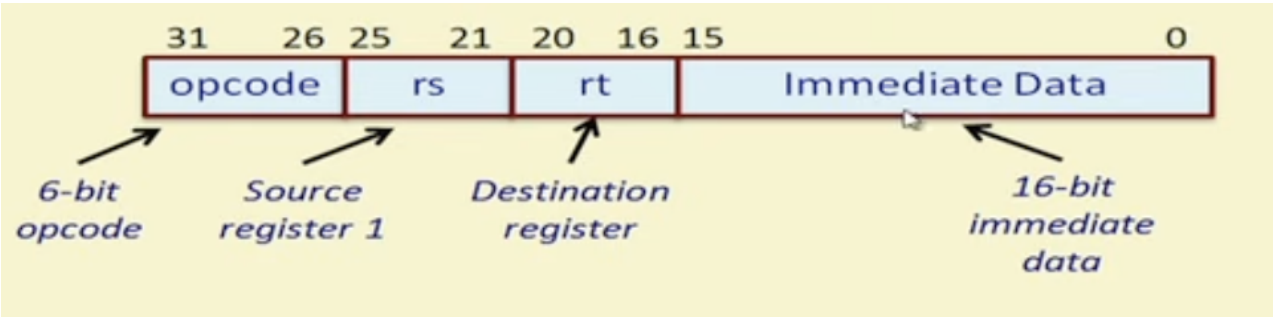
BEQZ  R1, Loop         // branch to Loop if R1 == 0
BNEQZ R5, Label        // branch to Label if R5 != 0
J     Loop             // unconditional jump to Loop
HLT                    // halt execution
```



2.2 I-type instructions

I-type instructions contain a 16-bit immediate field and typically specify one source and one destination register. The field-layout diagram is retained below. In this model, the R-type and I-type opcode assignments are:

R-type instruction	Opcode	I-type instruction	Opcode
ADD	000000	LW	001000
SUB	000001	SW	001001
AND	000010	ADDI	001010
OR	000011	SUBI	001011
SLT	000100	SLTI	001100
MUL	000101	BNEQZ	001101
HLT	111111	BEQZ	001110



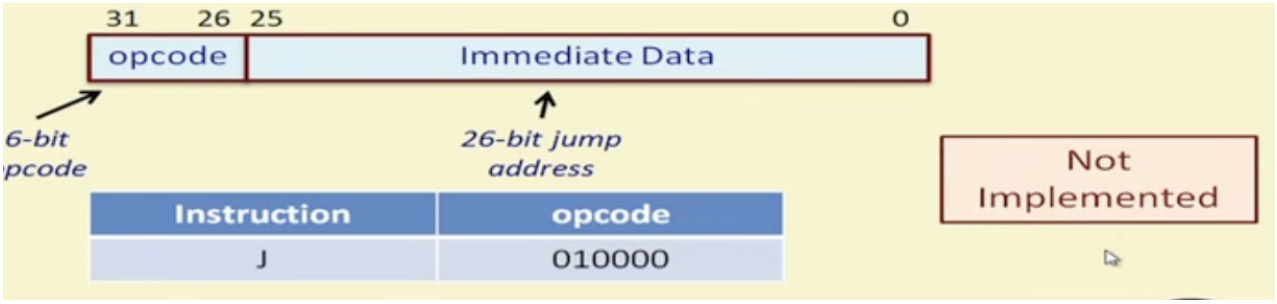
2.3 J-type instructions and addressing modes

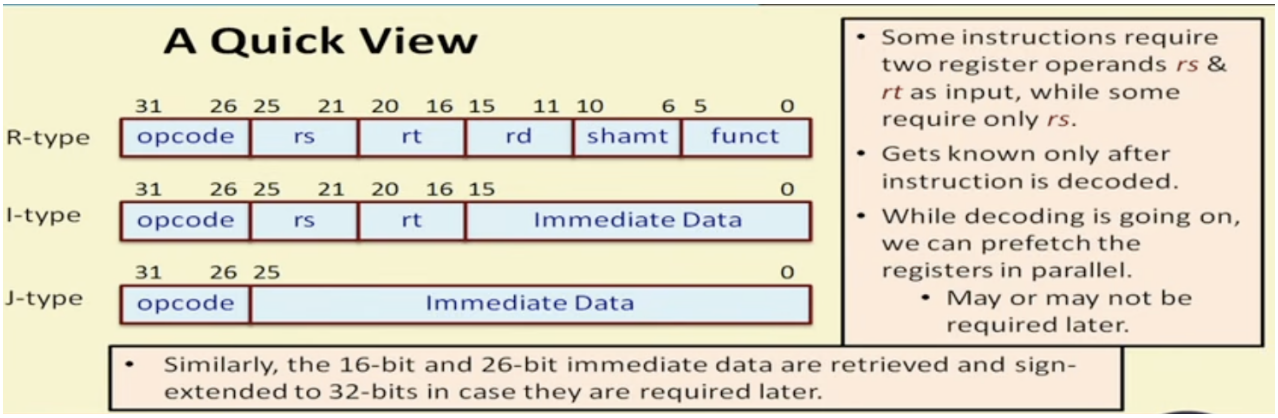
A J-type instruction contains a 26-bit target field. The target is extended to 28 bits by appending two zero bits on the right. The overall-format and addressing-mode figures are preserved. The following encoding examples are transcribed from the source note. The branch displacement is represented by a placeholder because its value depends on the label location.

Listing 3: Instruction encoding examples

```
LW R20, 84(R9)
    opcode  rs     rt     immediate
    001000  01001  10100  0000000001010100
    machine code: 0x21340054

BEQZ R25, Label
    opcode  rs     rt     displacement
    001110  11001  00000  YYYYYYYYYYYYYYYY
    machine code pattern: 0x3b20YYYY
```





3 MIPS instruction cycle

Instruction execution is described in five steps: instruction fetch (IF), instruction decode/register fetch (ID), execution or effective-address calculation (EX), memory access or branch completion (MEM), and register write-back (WB).

3.1 Fetch and decode

During IF, the instruction addressed by PC is fetched and the next sequential PC value is computed. The design holds that value in NPC so a branch can substitute its target before the PC is committed. During ID, the opcode and operand fields are decoded while the source registers are read; the fixed field positions allow these operations to occur in parallel. Immediate operands are sign-extended. The examples below summarize the addressing modes used in the instruction subset:

Listing 4: MIPS32 addressing-mode examples

```
Register addressing:         ADD  R1, R2, R3
```

```
Immediate addressing:        ADDI R1, R2, 200
Base addressing:             LW   R5, 150(R7)
PC-relative addressing:      BEQZ R3, Label
                             16-bit offset is added to PC
Pseudo-direct addressing:    J    Label
                             26-bit target field is used
```

The fetch and decode micro-operations are:

Listing 5: Instruction fetch and decode micro-operations

```
IF:
    IR  <= Mem[PC]
    NPC <= PC + 1

ID:
    A    <= Reg[rs]
    B    <= Reg[rt]
    Imm  <= sign_extend(IR[15:0])
    Imm1 <= zero_extend(IR[25:0])
```

Here, A, B, Imm, and Imm1 are temporary values used by later stages.

3.2 Execute, memory, and write-back

EX uses the ALU to perform the operation selected during decode. Depending on the instruction, it calculates an arithmetic/logical result, an effective address, or a branch target and condition. MEM is used by loads, stores, and branches. WB writes an ALU result or loaded memory value to the destination register. A store completes in MEM and therefore has no register write-back operation. The execution-stage operation depends on the instruction class:

Listing 6: Execute-stage operation by instruction class

```
Memory reference:           ALUOut <= A + Imm
                            Example: LW R3, 100(R8)

Register-register ALU:      ALUOut <= A func B
                            Example: SUB R2, R5, R12

Register-immediate ALU:     ALUOut <= A func Imm
                            Example: SUBI R2, R5, 524

Branch:                     ALUOut <= NPC + Imm
                            cond   <= (A == 0)
                            Example: BEQZ R2, Label
```

Loads, stores, and other instructions complete as follows:

Listing 7: Memory-stage and PC operations

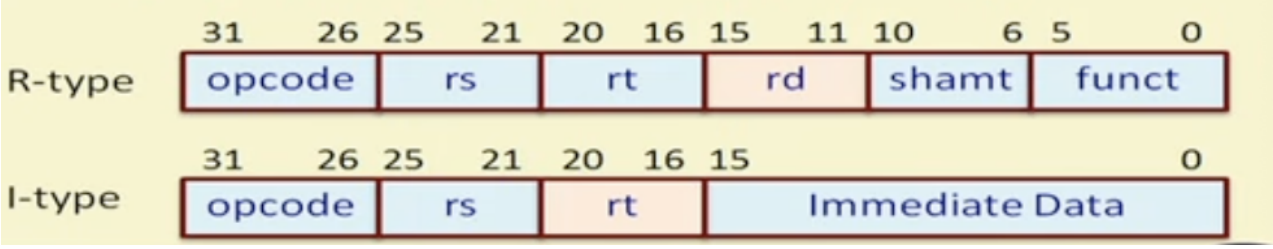
```
Load:
    PC  <= NPC
    LMD <= Mem[ALUOut]

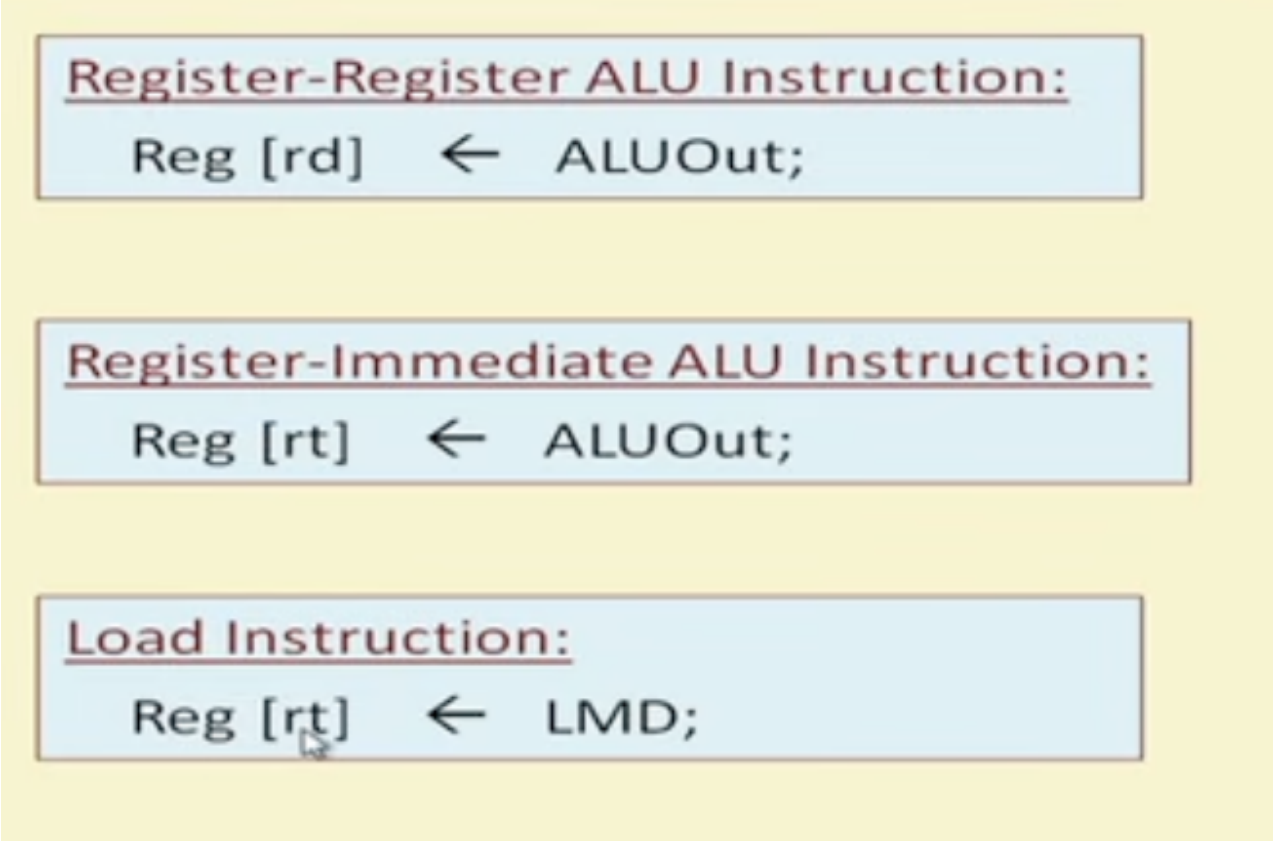
Store:
    PC           <= NPC
    Mem[ALUOut]  <= B

Branch:
    if (cond) PC <= ALUOut
    else      PC <= NPC

Other instructions:
    PC <= NPC
```

The following figure compares the R-type and I-type instruction fields.





The pipeline stage sequence for representative register-register and immediate operations is shown in editable form:

Listing 8: Representative arithmetic instruction micro-operations

```
ADD R2, R5, R10
    IF:  IR <= Mem[PC]; NPC <= PC + 1
    ID:  A <= Reg[rs]; B <= Reg[rt]
    EX:  ALUOut <= A + B
    MEM: PC <= NPC
    WB:  Reg[rd] <= ALUOut

ADDI R2, R5, 150
    IF:  IR <= Mem[PC]; NPC <= PC + 1
    ID:  A <= Reg[rs]; Imm <= sign_extend(IR[15:0])
    EX:  ALUOut <= A + Imm
    MEM: PC <= NPC
    WB:  Reg[rt] <= ALUOut
```

Load and store examples use the same effective-address calculation but differ in the MEM and WB stages:

Listing 9: Load and store instruction micro-operations

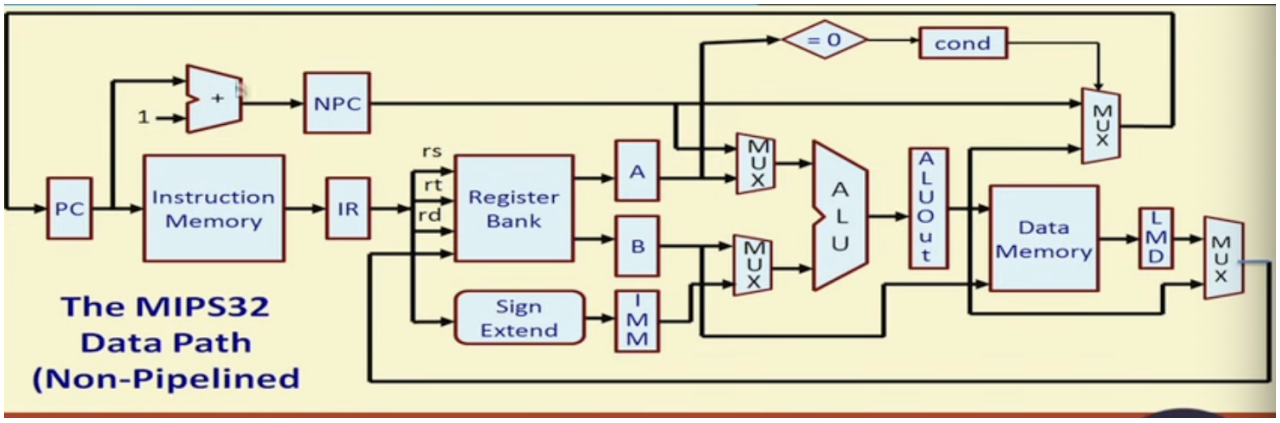
```
LW R2, 200(R6)
     IF:  IR <= Mem[PC]; NPC <= PC + 1
     ID:  A <= Reg[rs]; Imm <= sign_extend(IR[15:0])
     EX:  ALUOut <= A + Imm
     MEM: PC <= NPC; LMD <= Mem[ALUOut]
     WB:  Reg[rt] <= LMD

SW R3, 25(R10)
     IF:  IR <= Mem[PC]; NPC <= PC + 1
     ID:  A <= Reg[rs]; B <= Reg[rt]
          Imm <= sign_extend(IR[15:0])
     EX:  ALUOut <= A + Imm
     MEM: PC <= NPC; Mem[ALUOut] <= B
     WB:  no register write
```

For a conditional branch, the target and branch condition are computed in EX, then the PC is selected in MEM:

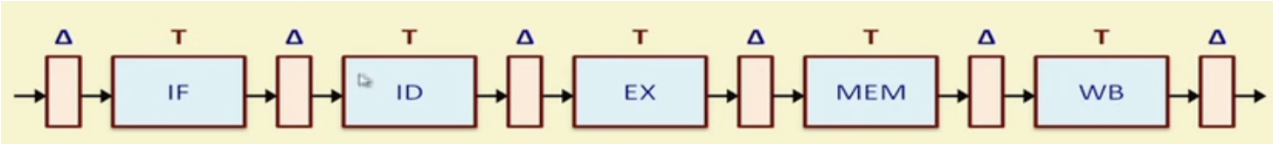
Listing 10: BEQZ instruction micro-operations

```
BEQZ R3, Label
     IF:  IR <= Mem[PC]; NPC <= PC + 1
     ID:  A <= Reg[rs]; B <= Reg[rt]
          Imm <= sign_extend(IR[15:0])
     EX:  ALUOut <= NPC + Imm
          cond <= (A == 0)
     MEM: if (cond) PC <= ALUOut
          else      PC <= NPC
     WB:  no register write
```



4 Pipelining the MIPS32 datapath

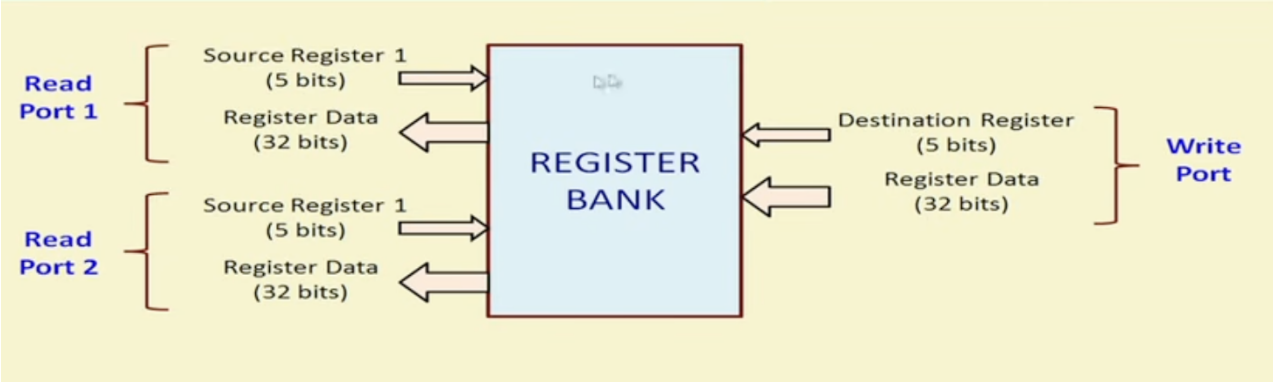
Pipelining divides the five instruction-cycle steps into stages so a new instruction can begin each clock cycle. Each stage must complete within one clock cycle. Temporary datapath values are carried in inter-stage latches: IF_ID, ID_EX, EX_MEM, and MEM_WB. For example, ID_EX_A is operand A held in the ID/EX latch. The instruction register is passed along with the values so each stage can identify the instruction it is processing.

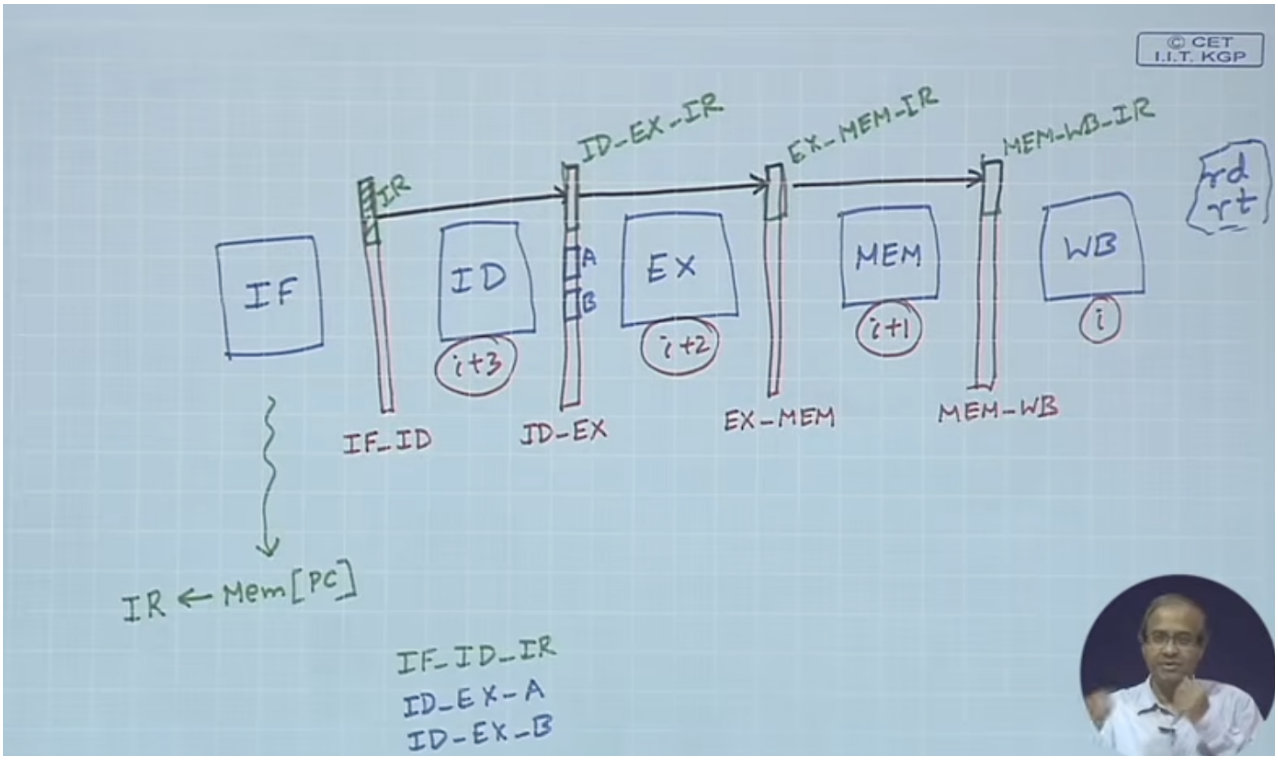


In the timing table, each successive instruction enters IF one cycle later. Once the pipeline is full, an instruction completes in WB every cycle:

Instruction	1	2	3	4	5	6	7	8
i	IF	ID	EX	MEM	WB			
$i+1$		IF	ID	EX	MEM	WB		
$i+2$			IF	ID	EX	MEM	WB	
$i+3$				IF	ID	EX	MEM	WB

Thus instructions i, $i+1$, and $i+2$ finish in cycles 5, 6, and 7, respectively.





The fetch-stage values are captured in IF/ID. A resolved taken branch selects its target; otherwise fetch advances sequentially:

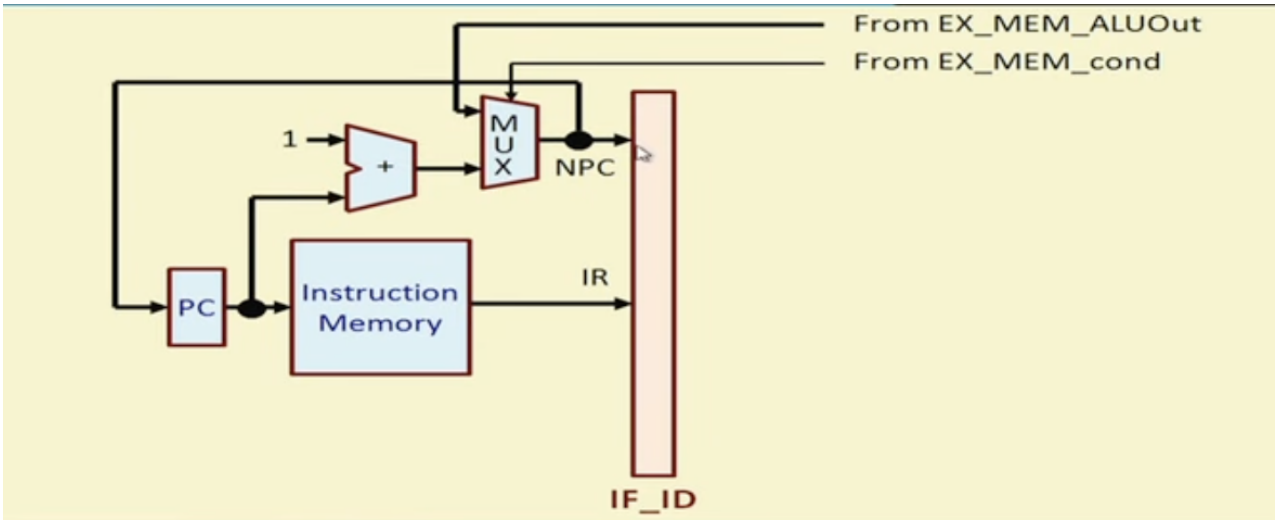
Listing 11: IF/ID latch and next-PC selection

```
IF_ID_IR <= Mem[PC]
```

```
2
3 if ((EX_MEM_IR.opcode == branch) && EX_MEM_cond)
4     IF_ID_NPC, PC <= EX_MEM_ALUOut
5 else
6     IF_ID_NPC, PC <= PC + 1
```

4.1 Pipeline stage operations and hardware

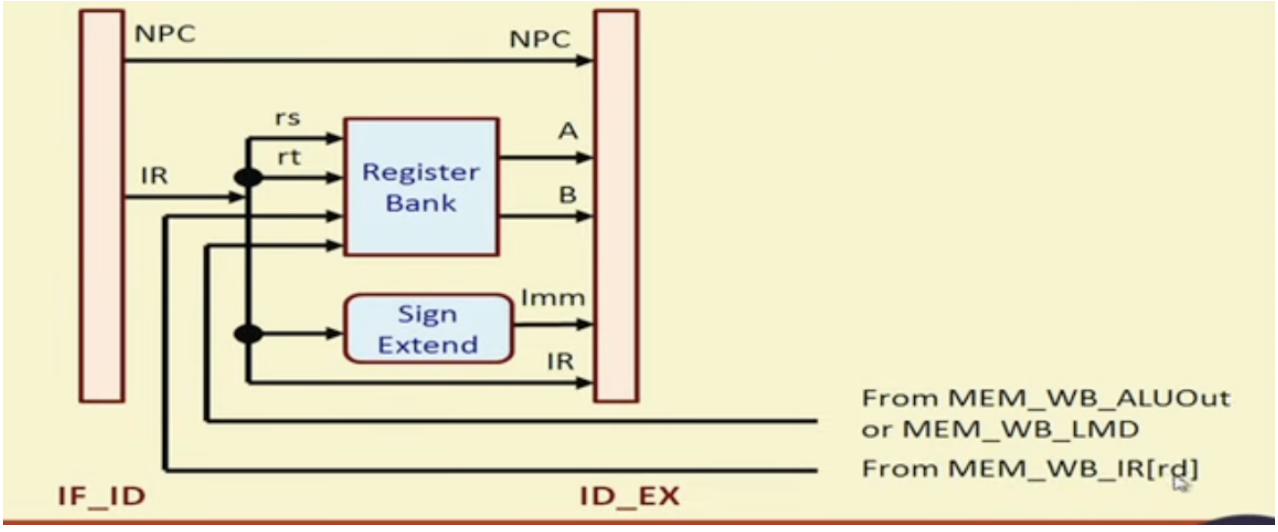
The original stage-by-stage micro-operation and hardware diagrams are retained. They show instruction fetch, decode, execute, memory, and write-back values moving between the stage latches.



The ID stage reads the source operands and prepares the values needed later in the pipeline:

Listing 12: ID-stage pipeline-register transfers

```
1 ID_EX_A   <= Reg[IF_ID_IR[rs]]
2 ID_EX_B   <= Reg[IF_ID_IR[rt]]
3 ID_EX_NPC <= IF_ID_NPC
4 ID_EX_IR  <= IF_ID_IR
5 ID_EX_Imm <= sign_extend(IF_ID_IR[15:0])
```



The EX stage computes ALU results, effective addresses, or branch conditions and carries the instruction forward:

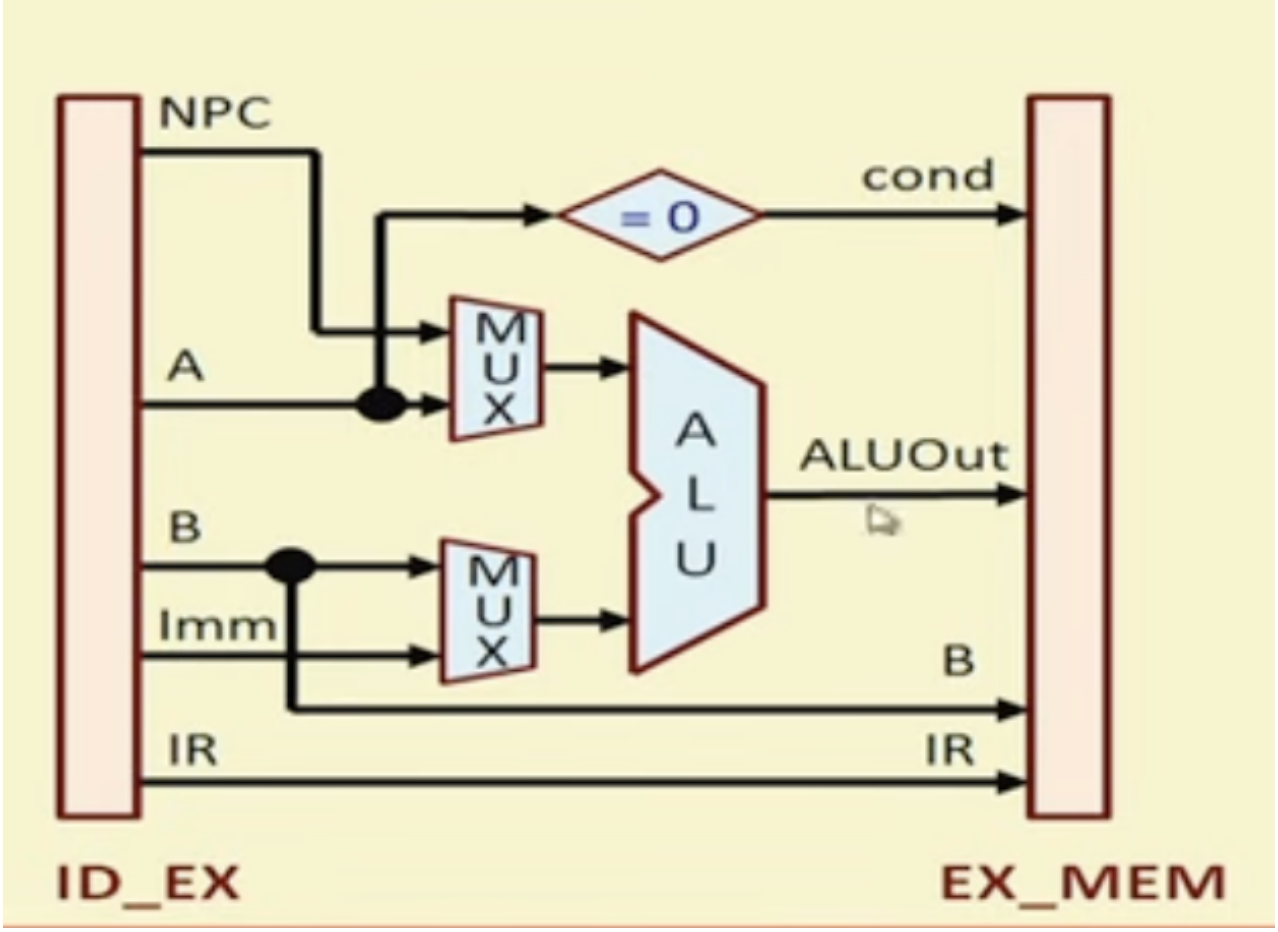
Listing 13: EX-stage pipeline-register transfers

```
R-R ALU:
    EX_MEM_IR     <= ID_EX_IR
    EX_MEM_ALUOut <= ID_EX_A func ID_EX_B

R-M ALU:
    EX_MEM_IR     <= ID_EX_IR
    EX_MEM_ALUOut <= ID_EX_A func ID_EX_Imm

Load or store:
    EX_MEM_IR     <= ID_EX_IR
    EX_MEM_ALUOut <= ID_EX_A + ID_EX_Imm
    EX_MEM_B      <= ID_EX_B

Branch:
    EX_MEM_ALUOut <= ID_EX_NPC + ID_EX_Imm
    EX_MEM_cond   <= (ID_EX_A == 0)
```



In MEM, ALU results are forwarded, loads read memory, and stores write memory:

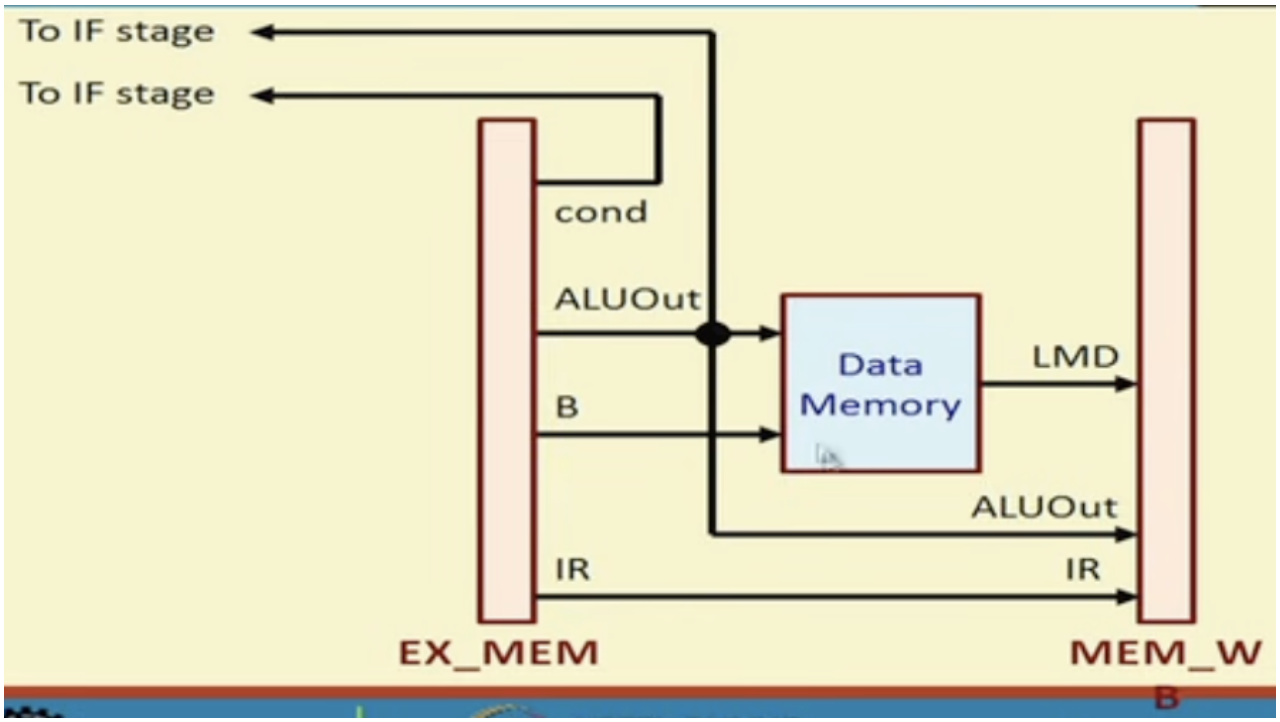
Listing 14: MEM-stage pipeline-register transfers

```
ALU:
    MEM_WB_IR     <= EX_MEM_IR
    MEM_WB_ALUOut <= EX_MEM_ALUOut

```

```
Load:
    MEM_WB_IR  <= EX_MEM_IR
    MEM_WB_LMD <= Mem[EX_MEM_ALUOut]

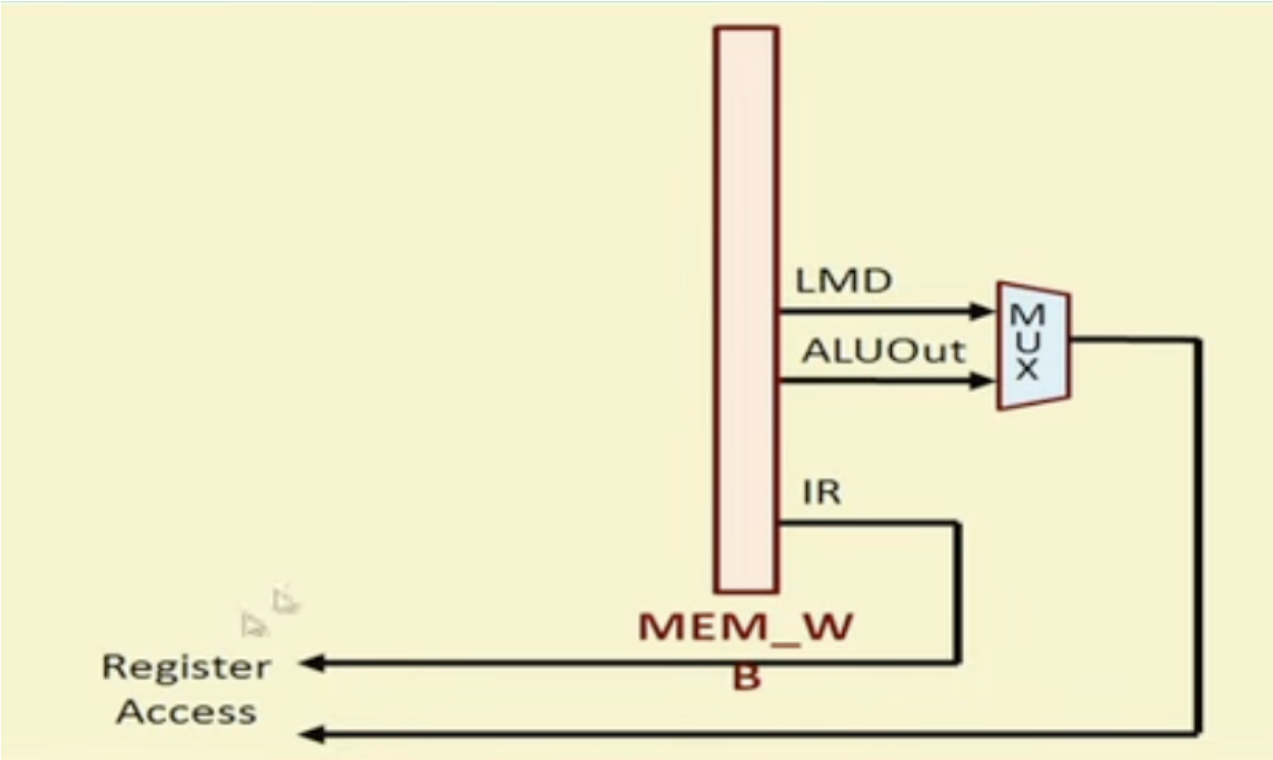
Store:
    MEM_WB_IR          <= EX_MEM_IR
    Mem[EX_MEM_ALUOut] <= EX_MEM_B
```



The WB stage writes ALU results or loaded data to the register selected by the instruction:

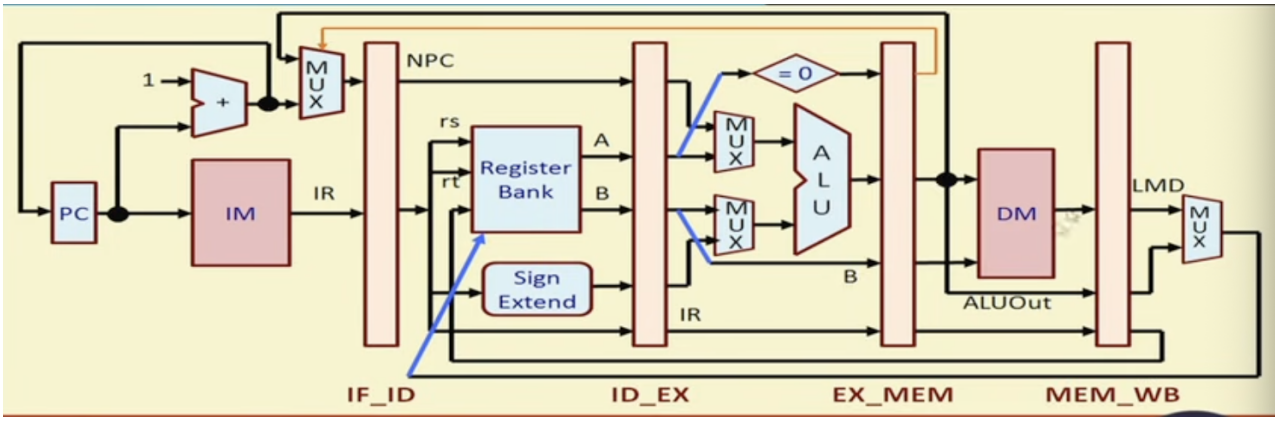
Listing 15: WB-stage register writes

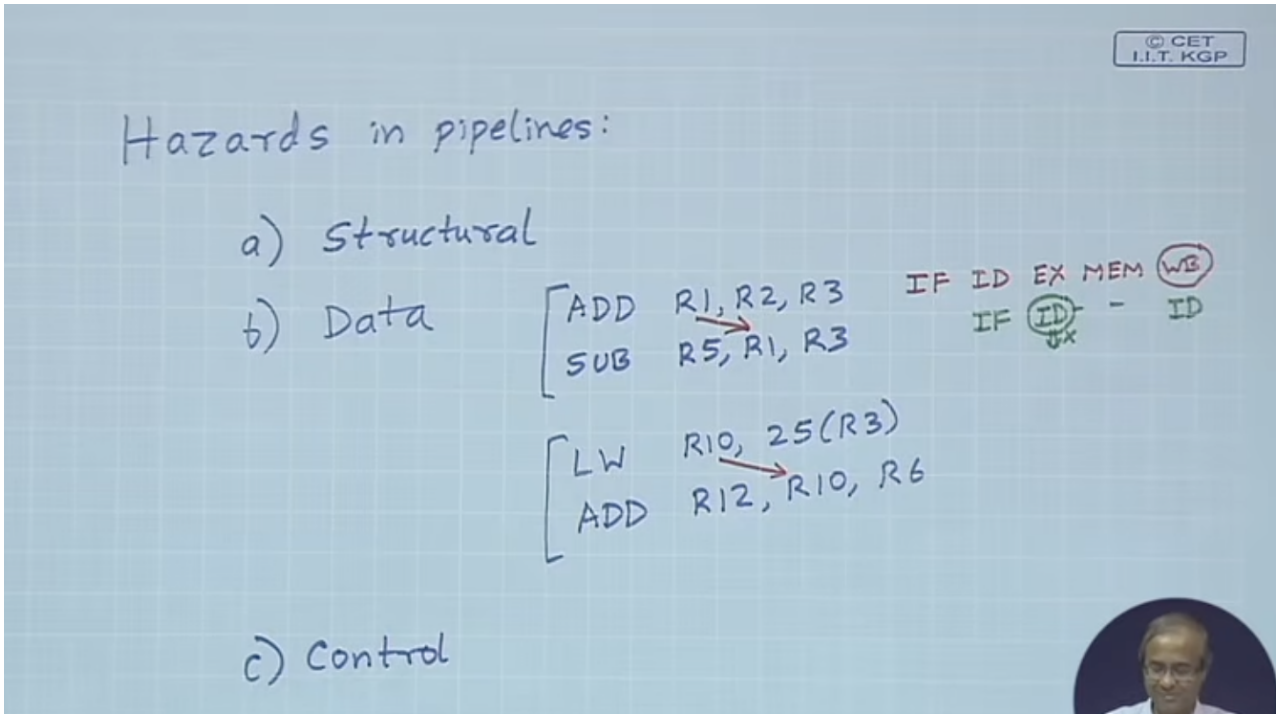
```
R-R ALU: Reg[MEM_WB_IR[rd]] <= MEM_WB_ALUOut
R-M ALU: Reg[MEM_WB_IR[rt]] <= MEM_WB_ALUOut
Load:    Reg[MEM_WB_IR[rt]] <= MEM_WB_LMD
```



4.2 Pipeline hazards

A hazard can prevent an instruction from proceeding in its intended clock cycle. The original notes distinguish structural, data, and control hazards. The later examples insert dummy instructions between dependent operations; this is a simple way to avoid the shown data hazards, not a general hazard-management unit.





5 Behavioral Verilog implementation

The behavioral model uses two clock phases and five pipeline stages. `HALTED` is set when the HLT instruction reaches write-back. `TAKEN_BRANCH` marks a resolved branch so instructions already in the pipeline cannot make unintended state changes. The code is included inline so the document builds without external source files.

Listing 16: Behavioral MIPS32 pipeline module

```
module pipe_MIPS32 (clk1, clk2);
input clk1, clk2; // Two-phase clock

reg [31:0] PC, IF_ID_IR, IF_ID_NPC;
reg [31:0] ID_EX_IR, ID_EX_NPC, ID_EX_A, ID_EX_B, ID_EX_Imm;
reg [2:0] ID_EX_type, EX_MEM_type, MEM_WB_type;
reg [31:0] EX_MEM_IR, EX_MEM_ALUout, EX_MEM_B;
reg EX_MEM_cond;
reg [31:0] MEM_WB_IR, MEM_WB_ALUout, MEM_WB_LMD;
reg [31:0] Reg [0:31]; // Register bank (32 x 32)
reg [31:0] Mem [0:1023]; // 1024 x 32 memory

parameter ADD=6'b000000, SUB=6'b000001, AND=6'b000010, OR=6'b000011,
          SLT=6'b000100, MUL=6'b000101, HLT=6'b111111, LW=6'b001000,
          SW=6'b001001, ADDI=6'b001010, SUBI=6'b001011, SLTI=6'b001100,
          BNEQZ=6'b001101, BEQZ=6'b001110;

parameter RR_ALU=3'b000, RM_ALU=3'b001, LOAD=3'b010, STORE=3'b011,
          BRANCH=3'b100, HALT=3'b101;

reg HALTED; // Set after HLT instruction completes in WB
reg TAKEN_BRANCH; // Disable instructions after a branch is taken

always @(posedge clk1) begin // IF stage
    if (HALTED == 0) begin
        if (((EX_MEM_IR[31:26] == BEQZ) && (EX_MEM_cond == 1)) ||
            ((EX_MEM_IR[31:26] == BNEQZ) && (EX_MEM_cond == 0))) begin
            IF_ID_IR <= #2 Mem[EX_MEM_ALUout];
```

```
            TAKEN_BRANCH <= #2 1'b1;
            IF_ID_NPC <= #2 EX_MEM_ALUout + 1;
            PC <= #2 EX_MEM_ALUout + 1;
        end else begin
            IF_ID_IR <= #2 Mem[PC];
            IF_ID_NPC <= #2 PC + 1;
            PC <= #2 PC + 1;
        end
    end
end

always @(posedge clk2) begin // ID stage
    if (HALTED == 0) begin
        if (IF_ID_IR[25:21] == 5'b00000)
            ID_EX_A <= 0;
        else
            ID_EX_A <= #2 Reg[IF_ID_IR[25:21]]; // rs

        if (IF_ID_IR[20:16] == 5'b00000)
            ID_EX_B <= 0;
        else
            ID_EX_B <= #2 Reg[IF_ID_IR[20:16]]; // rt

        ID_EX_NPC <= #2 IF_ID_NPC;
        ID_EX_IR <= #2 IF_ID_IR;
        ID_EX_Imm <= #2 {{16{IF_ID_IR[15]}}, (IF_ID_IR[15:0])};

        case (IF_ID_IR[31:26])
            ADD, SUB, AND, OR, SLT, MUL: ID_EX_type <= #2 RR_ALU;
            ADDI, SUBI, SLTI:           ID_EX_type <= #2 RM_ALU;
            LW:                         ID_EX_type <= #2 LOAD;
            SW:                         ID_EX_type <= #2 STORE;
            BNEQZ, BEQZ:                ID_EX_type <= #2 BRANCH;
            HLT:                        ID_EX_type <= #2 HALT;
            default:                    ID_EX_type <= #2 HALT; // Invalid opcode
        endcase
    end
end

always @(posedge clk1) begin // EX stage
    if (HALTED == 0) begin
        EX_MEM_type <= #2 ID_EX_type;
        EX_MEM_IR <= #2 ID_EX_IR;
        TAKEN_BRANCH <= #2 0;

        case (ID_EX_type)
            RR_ALU: begin
                case (ID_EX_IR[31:26]) // opcode
                    ADD: EX_MEM_ALUout <= #2 ID_EX_A + ID_EX_B;
                    SUB: EX_MEM_ALUout <= #2 ID_EX_A - ID_EX_B;
                    AND: EX_MEM_ALUout <= #2 ID_EX_A & ID_EX_B;
                    OR:  EX_MEM_ALUout <= #2 ID_EX_A | ID_EX_B;
                    SLT: EX_MEM_ALUout <= #2 ID_EX_A < ID_EX_B;
                    MUL: EX_MEM_ALUout <= #2 ID_EX_A * ID_EX_B;
                    default: EX_MEM_ALUout <= #2 32'hxxxxxxxx;
                endcase
            end
            RM_ALU: begin
                case (ID_EX_IR[31:26]) // opcode
                    ADDI: EX_MEM_ALUout <= #2 ID_EX_A + ID_EX_Imm;
                    SUBI: EX_MEM_ALUout <= #2 ID_EX_A - ID_EX_Imm;
                    SLTI: EX_MEM_ALUout <= #2 ID_EX_A < ID_EX_Imm;
                    default: EX_MEM_ALUout <= #2 32'hxxxxxxxx;
                endcase
            end
            LOAD, STORE: begin
                EX_MEM_ALUout <= #2 ID_EX_A + ID_EX_Imm;
                EX_MEM_B <= #2 ID_EX_B;
            end
            BRANCH: begin
                EX_MEM_ALUout <= #2 ID_EX_NPC + ID_EX_Imm;
```

```
                EX_MEM_cond <= #2 (ID_EX_A == 0);
            end
        endcase
    end
end

always @(posedge clk2) begin // MEM stage
    if (HALTED == 0) begin
        MEM_WB_type <= #2 EX_MEM_type;
        MEM_WB_IR <= #2 EX_MEM_IR;

        case (EX_MEM_type)
            RR_ALU, RM_ALU: MEM_WB_ALUout <= #2 EX_MEM_ALUout;
            LOAD: MEM_WB_LMD <= #2 Mem[EX_MEM_ALUout];
            STORE: begin
                if (TAKEN_BRANCH == 0) // Disable write
                    Mem[EX_MEM_ALUout] <= #2 EX_MEM_B;
            end
        endcase
    end
end

always @(posedge clk1) begin // WB stage
    if (TAKEN_BRANCH == 0) begin // Disable write if branch taken
        case (MEM_WB_type)
            RR_ALU: Reg[MEM_WB_IR[15:11]] <= #2 MEM_WB_ALUout; // rd
            RM_ALU: Reg[MEM_WB_IR[20:16]] <= #2 MEM_WB_ALUout; // rt
            LOAD:   Reg[MEM_WB_IR[20:16]] <= #2 MEM_WB_LMD; // rt
            HALT:   HALTED <= #2 1'b1;
        endcase
    end
end
endmodule
```

6 Test benches and examples

A test bench loads instructions into memory, initializes the PC and processor control flags, generates the two-phase clock, and checks the resulting register or memory values. Text-heavy instruction tables and simulation outputs are transcribed below; architecture and datapath diagrams remain images. The assembly and test-bench code is presented as editable text.

6.1 Example 1: add register values

The example initializes R1 and R2 with 10 and 20, then computes R4=R1+R2 and R5=R4+R3. The source slide says the third value is 30, but the actual program loads 25; therefore R4 is 30 and the final sum in R5 is 55. The binary encodings below follow the displayed assembly instructions.

Listing 17: Example 1 assembly and machine-code fields

```
Assembly            Machine code (binary fields)
ADDI R1, R0, 10     001010 00000 00001 0000000000001010
ADDI R2, R0, 20     001010 00000 00010 0000000000010100
ADDI R3, R0, 25     001010 00000 00011 0000000000011001
ADD  R4, R1, R2     000000 00001 00010 00100 00000 000000
ADD  R5, R4, R3     000000 00100 00011 00101 00000 000000
HLT                 111111 00000 00000 00000 00000 000000
```

Assembly program

Listing 18: Example 1 assembly instructions

```
ADDI R1, R0, 10
ADDI R2, R0, 20
ADDI R3, R0, 25
ADD  R4, R1, R2
ADD  R5, R4, R3
HLT
```

Verilog test bench

Listing 19: Example 1 test bench

```
module test_mips32;
reg clk1, clk2;
integer k;
pipe_MIPS32 mips (clk1, clk2);

initial begin
    clk1 = 0;
    clk2 = 0;
    repeat (20) begin
        #5 clk1 = 1;
        #5 clk1 = 0;
        #5 clk2 = 1;
        #5 clk2 = 0;
    end
end

initial begin
    for (k = 0; k < 31; k = k + 1)
        mips.Reg[k] = k;

    mips.Mem[0] = 32'h2801000a; // ADDI R1, R0, 10
    mips.Mem[1] = 32'h28020014; // ADDI R2, R0, 20
    mips.Mem[2] = 32'h28030019; // ADDI R3, R0, 25
    mips.Mem[3] = 32'h0ce77800; // OR R7,R7,R7 -- dummy instr.
    mips.Mem[4] = 32'h0ce77800; // OR R7,R7,R7 -- dummy instr.
    mips.Mem[5] = 32'h00222000; // ADD R4, R1, R2
    mips.Mem[6] = 32'h0ce77800; // OR R7,R7,R7 -- dummy instr.
    mips.Mem[7] = 32'h00832800; // ADD R5, R4, R3
    mips.Mem[8] = 32'hfc000000; // HLT

    mips.HALTED = 0;
    mips.PC = 0;
    mips.TAKEN_BRANCH = 0;
    #280;
    for (k = 0; k < 6; k = k + 1)
        $display("R%1d - %2d", k, mips.Reg[k]);
end

initial begin
    $dumpfile("mips.vcd");
    $dumpvars(0, test_mips32);
    #300 $finish;
end
endmodule
```

Simulation output

```
R0 - 0
R1 - 10
R2 - 20
R3 - 25
R4 - 30
R5 - 55
```

6.2 Example 2: load, add, and store

This program reads the word at memory location 120, adds 45, and stores the result at location 121. The input word is 85, so the expected stored value is 130. The steps are: place address 120 in R1, load its word into R2, add 45, then store R2 at address 121. The displayed encodings are transcribed here:

Listing 20: Example 2 assembly and machine-code fields

```
Assembly             Machine code (binary fields)
ADDI R1, R0, 120     001010 00000 00001 0000000001111000
LW   R2, 0(R1)       001000 00001 00010 0000000000000000
ADDI R2, R2, 45      001010 00010 00010 0000000000101101
SW   R2, 1(R1)       001001 00001 00010 0000000000000001
HLT                  111111 00000 00000 00000 00000 000000
```

Assembly program

Listing 21: Example 2 assembly instructions

```
ADDI R1, R0, 120
LW   R2, 0(R1)
ADDI R2, R2, 45
SW   R2, 1(R1)
HLT
```

Verilog test bench

Listing 22: Example 2 test bench

```
module test_mips32;
reg clk1, clk2;
integer k;
pipe_MIPS32 mips (clk1, clk2);

initial begin
    clk1 = 0;
    clk2 = 0;
    repeat (50) begin
        #5 clk1 = 1;
        #5 clk1 = 0;
        #5 clk2 = 1;
        #5 clk2 = 0;
    end
end

initial begin
    for (k = 0; k < 31; k = k + 1)
        mips.Reg[k] = k;

    mips.Mem[0] = 32'h28010078; // ADDI R1, R0, 120
    mips.Mem[1] = 32'h0c631800; // OR R3,R3,R3 -- dummy instr.
    mips.Mem[2] = 32'h20220000; // LW R2, 0(R1)
    mips.Mem[3] = 32'h0c631800; // OR R3,R3,R3 -- dummy instr.
    mips.Mem[4] = 32'h2842002d; // ADDI R2, R2, 45
    mips.Mem[5] = 32'h0c631800; // OR R3,R3,R3 -- dummy instr.
    mips.Mem[6] = 32'h24220001; // SW R2, 1(R1)
    mips.Mem[7] = 32'hfc000000; // HLT
    mips.Mem[120] = 85;

    mips.PC = 0;
    mips.HALTED = 0;
    mips.TAKEN_BRANCH = 0;
    #500;
    $display("Mem[120]: %4d \nMem[121]: %4d",
             mips.Mem[120], mips.Mem[121]);
end

initial begin
```

```
    $dumpfile("mips.vcd");
    $dumpvars(0, test_mips32);
    #600 $finish;
end
endmodule
```

Simulation output

```
Mem[120]: 85
Mem[121]: 130
```

6.3 Example 3: factorial

The third program computes the factorial of the value at memory location 200. R10 holds address 200, R3 holds the loaded input, and R2 starts at 1. The loop multiplies R2 by R3, decrements R3, and repeats until R3 is zero; it then stores R2 at address 198 using offset -2. With input 7, the result is 5040. The binary branch offset below is -4, matching the test-bench machine code and target instruction address.

Listing 23: Example 3 assembly and machine-code fields

```
Assembly            Machine code (binary fields)
ADDI R10, R0, 200   001010 00000 01010 0000000011001000
ADDI R2, R0, 1      001010 00000 00010 0000000000000001
LW   R3, 0(R10)     001000 01010 00011 0000000000000000
MUL  R2, R2, R3     000101 00010 00011 00010 00000 000000
SUBI R3, R3, 1      001011 00011 00011 0000000000000001
BNEQZ R3, Loop      001101 00011 00000 1111111111111100
SW   R2, -2(R10)    001001 01010 00010 1111111111111110
HLT                 111111 00000 00000 00000 00000 000000
```

Assembly program

Listing 24: Example 3 assembly instructions

```
ADDI  R10, R0, 200
ADDI  R2, R0, 1
LW    R3, 0(R10)
Loop: MUL   R2, R2, R3
      SUBI  R3, R3, 1
      BNEQZ R3, Loop
      SW    R2, -2(R10)
HLT
```

Verilog test bench

Listing 25: Example 3 test bench

```
module test_mips32;
reg clk1, clk2;
integer k;
pipe_MIPS32 mips (clk1, clk2);

initial begin
    clk1 = 0;
    clk2 = 0;
    repeat (50) begin
        #5 clk1 = 1;
        #5 clk1 = 0;
        #5 clk2 = 1;
        #5 clk2 = 0;
    end
end

```

```
initial begin
    for (k = 0; k < 31; k = k + 1)
        mips.Reg[k] = k;

    mips.Mem[0]  = 32'h280a00c8; // ADDI R10, R0, 200
    mips.Mem[1]  = 32'h28020001; // ADDI R2, R0, 1
    mips.Mem[2]  = 32'h0e94a000; // OR R20,R20,R20 -- dummy instr.
    mips.Mem[3]  = 32'h21430000; // LW R3, 0(R10)
    mips.Mem[4]  = 32'h0e94a000; // OR R20,R20,R20 -- dummy instr.
    mips.Mem[5]  = 32'h14431000; // Loop: MUL R2, R2, R3
    mips.Mem[6]  = 32'h2c630001; // SUBI R3, R3, 1
    mips.Mem[7]  = 32'h0e94a000; // OR R20,R20,R20 -- dummy instr.
    mips.Mem[8]  = 32'h3460fffc; // BNEQZ R3, Loop (offset -4)
    mips.Mem[9]  = 32'h2542fffe; // SW R2, -2(R10)
    mips.Mem[10] = 32'hfc000000; // HLT
    mips.Mem[200] = 7;

    mips.PC = 0;
    mips.HALTED = 0;
    mips.TAKEN_BRANCH = 0;
    #2000;
    $display("Mem[200] = %2d, Mem[198] = %6d",
             mips.Mem[200], mips.Mem[198]);
end

initial begin
    $dumpfile("mips.vcd");
    $dumpvars(0, test_mips32);
    $monitor("R2: %4d", mips.Reg[2]);
    #3000 $finish;
end
endmodule
```

Simulation output

```
R2: 2
R2: 1
R2: 7
R2: 42
R2: 210
R2: 840
R2: 2520
R2: 5040
R2: 5040
Mem[200] = 7, Mem[198] = 5040
```

7 Summary and limitations

The examples demonstrate a behavioral model of a pipelined MIPS32 processor and simple test benches for arithmetic, load/store, and branch behavior. The original notes do not implement a general hazard-avoidance mechanism; dummy instructions are inserted between dependent instructions in the examples. A synthesizable hardware design would typically describe the pipeline stages structurally, with control logic generating the datapath signals in the required sequence.